

Types in R & Python

Lecture 02

Dr. Colin Rundel

```
1 import math
```

py

Type systems

Dynamic typing

Both R and Python are *dynamically typed* languages - the type of a value is determined at runtime and a variable (name) is not restricted to a single type.

```
1 x = 1
2 typeof(x)
```

R

```
[1] "double"
```

```
1 x = "one"
2 typeof(x)
```

R

```
[1] "character"
```

```
1 x = 1
2 type(x)
```

py

```
<class 'int'>
```

```
1 x = "one"
2 type(x)
```

py

```
<class 'str'>
```

Where the languages differ is in their fundamental unit of data,

- in R (almost) everything is a *vector*
- in Python individual values are separate *objects* that can be grouped into *container* objects

Dynamic describes when types are checked; it does not mean that values lack types or that every operation between

Vectors in R

The fundamental building block for data in R is the vector—a collection of related values.

R has two types of vectors:

- atomic vectors (*vectors*) — homogeneous collections with a single underlying type (e.g. `logical`, `integer`, `double`, or `character`).
- generic vectors (*lists*) — heterogeneous collections that can contain any R object, including other lists, allowing hierarchical or tree-like structures.

There are no scalars in R - a single value like `1` is just a vector of length 1.

```
1 length(1)
```

R

```
[1] 1
```

```
1 length(c(1, 2, 3))
```

R

```
[1] 3
```

Objects in Python

Python is built around an object-oriented class system—every value is an object with a type, including numbers, strings, and containers.

```
1 1
```

```
1
```

```
1 type(1)
```

```
<class 'int'>
```

```
1 "one"
```

```
'one'
```

```
1 type("one")
```

```
<class 'str'>
```

```
1 [1, 2, 3]
```

```
[1, 2, 3]
```

```
1 type([1, 2, 3])
```

```
<class 'list'>
```

Individual objects can be collected into general-purpose *containers* such as `list`, which can be heterogeneous and nested. Base Python has no direct equivalent to R's atomic vectors - that role is filled by `numpy` arrays (more on these next week).

Basic types

R's atomic types

R has six atomic vector types, we can check the type of any object using `typeof()`, `mode()` (a higher level grouping of types), or `class()`.

<code>typeof()</code>	<code>mode()</code>	<code>class()</code>	Example
logical	logical	logical	<code>TRUE</code>
double	numeric	numeric	<code>1.5, 2</code>
integer	numeric	integer	<code>1L, 1:3</code>
character	character	character	<code>"hello"</code>
complex	complex	complex	<code>1+2i</code>
raw	raw	raw	<code>as.raw(255)</code>

There are additional types in R, e.g. `list`, `closure`, `environment`, etc. which we will see in the coming lectures. See `?typeof` for more information.

Python's basic types

Python's common built-in value types are similar, but each value is an individual object rather than a vector.

Type	<code>type()</code>	Example
boolean	<code>bool</code>	<code>True</code>
integer	<code>int</code>	<code>1</code>
float	<code>float</code>	<code>1.5, 2.0</code>
complex	<code>complex</code>	<code>1+2j</code>
string	<code>str</code>	<code>"hello"</code>
none	<code>NoneType</code>	<code>None</code>

There is also `bytes` (similar to R's `raw`), as well as container types such as `list`, `dict`, and `set`. We will introduce lists later

Logical / boolean values

```
1 typeof(TRUE)
```

R

```
[1] "logical"
```

```
1 typeof(FALSE)
```

R

```
[1] "logical"
```

```
1 TRUE + TRUE
```

R

```
[1] 2
```

```
1 sum(c(TRUE, FALSE, TRUE))
```

R

```
[1] 2
```

```
1 type(True)
```

py

```
<class 'bool'>
```

```
1 type(False)
```

py

```
<class 'bool'>
```

```
1 True + True
```

py

```
2
```

```
1 sum([True, False, True])
```

py

```
2
```

In both languages boolean values behave like the integers `1` and `0` when used in arithmetic. In Python this is explicit - `bool` is a subclass of `int`.

Numeric values

```
1 typeof(1.5)
```

R

```
[1] "double"
```

```
1 typeof(7)
```

R

```
[1] "double"
```

```
1 typeof(7L)
```

R

```
[1] "integer"
```

```
1 typeof(1:3)
```

R

```
[1] "integer"
```

```
1 type(1.5)
```

py

```
<class 'float'>
```

```
1 type(7)
```

py

```
<class 'int'>
```

```
1 type(7.0)
```

py

```
<class 'float'>
```

```
1 type(7.)
```

py

```
<class 'float'>
```

The *default* numeric type differs between the languages - a bare literal like `7` is a `double` in R but an `int` in Python. In R, integer literals require an `L` suffix, in Python floats require a decimal point (or an exponent, e.g. `1e3`).

Integers

R integers are 32-bit and overflow to `NA` (with a warning), Python integers have arbitrary precision.

```
1 .Machine$integer.max
```

```
[1] 2147483647
```

```
1 .Machine$integer.max + 1L
```

```
Warning in .Machine$integer.max + 1L: NAs
```

```
[1] NA
```

```
1 .Machine$integer.max + 1
```

```
[1] 2147483648
```

```
1 2^100
```

```
[1] 1.267651e+30
```

```
1 2**31 - 1
```

```
2147483647
```

```
1 2**31
```

```
2147483648
```

```
1 2**100
```

```
1267650600228229401496703205376
```

In practice this rarely matters in R since the default numeric type is `double`, but it is one reason why R's `integer` type is used sparingly (e.g. for indexing and counts). Note that `numpy` and `pandas` behave more like R here, as they use fixed width integers (e.g. `int64`).

Floating-point precision

R's `double` and Python's `float` use binary floating-point representations, so many decimal values cannot be represented exactly.

```
1 0.1 + 0.2
```

R

```
[1] 0.3
```

```
1 0.1 + 0.2 == 0.3
```

R

```
[1] FALSE
```

```
1 all.equal(0.1 + 0.2, 0.3)
```

R

```
[1] TRUE
```

```
1 0.1 + 0.2
```

py

```
0.30000000000000004
```

```
1 0.1 + 0.2 == 0.3
```

py

```
False
```

```
1 import math
2 math.isclose(0.1 + 0.2, 0.3)
```

py

```
True
```

Complex numbers

Both languages have a built-in complex type - the main difference is the literal syntax, R uses `i` for the imaginary unit while Python uses `j` (the engineering convention).

```
1 z = 1+2i; typeof(z)
```

R

```
[1] "complex"
```

```
1 c(Re(z), Im(z), Mod(z))
```

R

```
[1] 1.000000 2.000000 2.236068
```

```
1 z = 1+2j; type(z)
```

py

```
<class 'complex'>
```

```
1 (z.real, z.imag, abs(z))
```

py

```
(1.0, 2.0, 2.23606797749979)
```

With a real negative input, R's `sqrt()` returns `NaN` while Python's `math.sqrt()` raises an error. R requires complex input, whereas Python's `**` operator promotes the result to complex.

```
1 sqrt(-1)
```

R

```
Warning in sqrt(-1): NaNs produced
```

```
[1] NaN
```

```
1 sqrt(-1+0i)
```

R

```
[1] 0+1i
```

```
1 (-1)^0.5
```

R

```
[1] NaN
```

```
1 math.sqrt(-1)
```

py

```
ValueError: expected a nonnegative input, got -1
```

```
1 sqrt(-1+0j)
```

py

```
NameError: name 'sqrt' is not defined
```

```
1 (-1) ** 0.5
```

py

```
(6.123233995736766e-17+1j)
```

Numeric promotion / coercion

Both languages *promote* to the more “general” type when mixing numeric types (`logical / bool` → `integer / int` → `double / float` → `complex`),

```
1 typeof(1L + 1.5) R
```

```
[1] "double"
```

```
1 typeof(TRUE + 1L) R
```

```
[1] "integer"
```

```
1 typeof(1L + 2i) R
```

```
[1] "complex"
```

```
1 type(1 + 1.5) py
```

```
<class 'float'>
```

```
1 type(True + 1) py
```

```
<class 'int'>
```

```
1 type(1 + 2j) py
```

```
<class 'complex'>
```

For integer inputs, division (`/`) returns a `double / float` rather than an integer in both languages,

```
1 4L / 2L R
```

```
[1] 2
```

```
1 typeof(4L / 2L) R
```

```
[1] "double"
```

```
1 4 / 2 py
```

```
2.0
```

```
1 type(4 / 2) py
```

```
<class 'float'>
```

Type predicates

```
1 is.integer(1)
```

R

```
[1] FALSE
```

```
1 is.integer(1L)
```

R

```
[1] TRUE
```

```
1 is.double(1)
```

R

```
[1] TRUE
```

```
1 is.numeric(1)
```

R

```
[1] TRUE
```

```
1 is.numeric(1L)
```

R

```
[1] TRUE
```

```
1 is.character("1")
```

R

```
[1] TRUE
```

```
1 isinstance(1, int)
```

py

```
True
```

```
1 isinstance(1.0, int)
```

py

```
False
```

```
1 isinstance(1.0, float)
```

py

```
True
```

```
1 isinstance(True, int)
```

py

```
True
```

```
1 isinstance("1", str)
```

py

```
True
```

`is.numeric()` checks the *mode* (`double` or `integer`) while the others check the *type*. Python's `isinstance()` respects class inheritance (hence `True` is an `int`).

Strings

String literals

```
1 typeof("hello")
```

R

```
[1] "character"
```

```
1 typeof('world')
```

R

```
[1] "character"
```

```
1 "abc'123"
```

R

```
[1] "abc'123"
```

```
1 'abc"123'
```

R

```
[1] "abc\"123"
```

```
1 "abc\"123"
```

R

```
[1] "abc\"123"
```

```
1 type("hello")
```

py

```
<class 'str'>
```

```
1 type('world')
```

py

```
<class 'str'>
```

```
1 "abc'123"
```

py

```
"abc'123"
```

```
1 'abc"123'
```

py

```
'abc"123'
```

```
1 "abc\"123"
```

py

```
'abc"123'
```

In both languages single or double quotes are fine (the opening and closing quote must match). Quote characters can be included by escaping or by using the non-matching quote.

Multi-line strings

R strings can contain newlines directly (or via `\n`), in Python a literal must be triple quoted (`"""` or `'''`) to span multiple lines.

```
1 x = "line one
2   line 'two'"
3 x
```

R

```
[1] "line one\nline 'two'"
```

```
1 cat(x)
```

R

```
line one
line 'two'
```

```
1 x = """line one
2   line 'two'"""
3 x
```

py

```
"line one\nline 'two'"
```

```
1 print(x)
```

py

```
line one
line 'two'
```

String interpolation

Python's f-strings (3.6+) evaluate arbitrary expressions inside `{}`. R has no built-in equivalent - `paste()` / `paste0()` are the base approach and the `glue` package provides similar `{}` interpolation.

```
1 x = c(1, 2, 3)
2 paste0("x has ", length(x), " elements")
```

R

```
[1] "x has 3 elements"
```

```
1 glue::glue("x has {length(x)} elements")
```

R

```
x has 3 elements
```

```
1 glue::glue("From {min(x)} to {max(x)}")
```

R

```
From 1 to 3
```

```
1 x = [1, 2, 3]
2 "x has " + str(len(x)) + " elements"
```

py

```
'x has 3 elements'
```

```
1 f"x has {len(x)} elements"
```

py

```
'x has 3 elements'
```

```
1 f"From {min(x)} to {max(x)}"
```

py

```
'From 1 to 3'
```

Coercion

Atomic vectors are homogeneous

R's atomic vectors must contain values of a single type, so combining different types with `c()` forces *coercion* to the most general type present,

```
1 c(1, "Hello")
```

```
[1] "1"      "Hello"
```

```
1 c(FALSE, 3L)
```

```
[1] 0 3
```

```
1 c(1.2, 3L)
```

```
[1] 1.2 3.0
```

```
1 c(FALSE, "Hello")
```

```
[1] "FALSE" "Hello"
```

Atomic vectors have flat underlying storage—nested calls to `c()` combine their contents rather than create a nested structure,

```
1 c(1, c(2, c(3)))
```

```
[1] 1 2 3
```

Base Python has no direct counterpart. Lists can mix types and be nested, like R's lists; `numpy` arrays are closer because they are homogeneous and choose a common data type. We will see these later.

Implicit coercion in R

R is quite liberal about coercion - builtin operators and functions (e.g. `+`, `&`, `log()`, etc.) will generally attempt to coerce values to an appropriate type for the given operation (numeric for math, logical for logical, etc.)

```
1 3.1 + 1L
```

```
[1] 4.1
```

```
1 5 + FALSE
```

```
[1] 5
```

```
1 log(TRUE)
```

```
[1] 0
```

```
1 TRUE & 7
```

```
[1] TRUE
```

```
1 FALSE | !5
```

```
[1] FALSE
```

```
1 TRUE == "TRUE"
```

```
[1] TRUE
```

```
1 1 == "1"
```

```
[1] TRUE
```

Arithmetic with strings is the exception, R will not coerce a character value to a number for math,

```
1 "1" + 1
```

```
Error in `"1" + 1`:  
! non-numeric argument to binary operator
```

Implicit coercion in Python

Python is more conservative: it promotes values within the numeric types (`bool` → `int` → `float` → `complex`) but generally does not implicitly convert between strings and numbers. Equality comparisons between these types return `False`, while unsupported ordering and arithmetic operations raise an error.

```
1 True + 1
```

py

2

```
1 5 + False
```

py

5

```
1 1 == True
```

py

True

```
1 (1+0j) == 1
```

py

True

```
1 1 == "1"
```

py

False

```
1 "abc" > 5
```

py

TypeError: '>' not supported between insta

```
1 "abc" + 5
```

py

TypeError: can only concatenate str (not "

```
1 "abc" + str(5)
```

py

'abc5'

Operator overloading in Python

Do be aware that Python has a number of useful / idiosyncratic overloads for some of the basic operators - depending on the types involved the same operator can do very different things.

```
1 "abc" + "def"
```

py

```
'abcdef'
```

```
1 "abc" * 3
```

py

```
'abcabcabc'
```

```
1 [1, 2] + [3, 4]
```

py

```
[1, 2, 3, 4]
```

```
1 [1, 2] * 2
```

py

```
[1, 2, 1, 2]
```

```
1 "abc" ** 2
```

py

```
TypeError: unsupported operand type(s) for
```

```
1 [1, 2] * [3, 4]
```

py

```
TypeError: can't multiply sequence by non-
```

Explicit coercion

R uses the `as.*()` family of functions for explicit conversion, Python uses the type constructors (`int()`, `float()`, `str()`, `bool()`, etc.),

```
1 as.logical(5.2)
```

R

```
[1] TRUE
```

```
1 as.integer(pi)
```

R

```
[1] 3
```

```
1 as.double("7.2")
```

R

```
[1] 7.2
```

```
1 as.integer("2.1")
```

R

```
[1] 2
```

```
1 as.integer("one")
```

R

Warning: NAs introduced by coercion

```
[1] NA
```

```
1 bool(5.2)
```

py

```
True
```

```
1 int(3.14159)
```

py

```
3
```

```
1 float("7.2")
```

py

```
7.2
```

```
1 int("2.1")
```

py

```
ValueError: invalid literal for int() with base
```

Coercion to logical / bool

```
1 as.logical(0)
```

R

```
[1] FALSE
```

```
1 as.logical("TRUE")
```

R

```
[1] TRUE
```

```
1 as.logical("T")
```

R

```
[1] TRUE
```

```
1 as.logical("yes")
```

R

```
[1] NA
```

```
1 as.logical("")
```

R

```
[1] NA
```

```
1 bool(0)
```

py

```
False
```

```
1 bool("False")
```

py

```
True
```

```
1 bool("")
```

py

```
False
```

```
1 bool([])
```

py

```
False
```

```
1 bool(None)
```

py

```
False
```

Python uses the idea of *truthiness* (0, 0.0, "", None, and empty containers are *falsy*, almost everything else is *truthy*) throughout the language, e.g. in conditionals. For character input, R recognizes only "TRUE", "true", "True", "T" (and their FALSE counterparts); other strings become NA.

Missing & special values

Missing values in R

R uses `NA` to represent missing values in its data structures.

What may not be obvious is that there are different `NA`s for the different atomic types.

<pre>1 NA</pre> <pre>[1] NA</pre>	<pre>1 typeof(NA)</pre> <pre>[1] "logical"</pre>	<pre>1 typeof(NA_character_)</pre> <pre>[1] "character"</pre>
<pre>1 NA+1</pre> <pre>[1] NA</pre>	<pre>1 typeof(NA+1)</pre> <pre>[1] "double"</pre>	<pre>1 typeof(NA_real_)</pre> <pre>[1] "double"</pre>
<pre>1 NA+1L</pre> <pre>[1] NA</pre>	<pre>1 typeof(NA+1L)</pre> <pre>[1] "integer"</pre>	<pre>1 typeof(NA_integer_)</pre> <pre>[1] "integer"</pre>
<pre>1 c(NA, "")</pre> <pre>[1] NA ""</pre>	<pre>1 typeof(c(NA, ""))</pre> <pre>[1] "character"</pre>	<pre>1 typeof(NA_complex_)</pre> <pre>[1] "complex"</pre>

NA stickiness

As **NA**s represent missing values (most) calculations using them return a missing value.

```
1 1 + NA
```

R

```
[1] NA
```

```
1 1 / NA
```

R

```
[1] NA
```

```
1 NA * 5
```

R

```
[1] NA
```

```
1 sqrt(NA)
```

R

```
[1] NA
```

```
1 3^NA
```

R

```
[1] NA
```

```
1 sum(c(1, 2, 3, NA))
```

R

```
[1] NA
```

Aggregation / summarization functions (e.g. `sum()`, `mean()`, `sd()`, etc.) will often have an `na.rm` argument which *removes* the missing values from the calculation.

```
1 sum(c(1, 2, 3, NA), na.rm = TRUE)
```

R

```
[1] 6
```

```
1 mean(c(1, 2, 3, NA), na.rm = TRUE)
```

R

```
[1] 2
```

NAs are not always sticky

A useful mental model for **NAs** is to consider them as an unknown value that could take any of the possible values for a given type.

For numbers or characters this isn't helpful, but for a logical value it must either be **TRUE** or **FALSE** which is relevant for certain calculations.

```
1 TRUE & NA
```

```
R
```

```
[1] NA
```

```
1 FALSE & NA
```

```
R
```

```
[1] FALSE
```

```
1 TRUE | NA
```

```
R
```

```
[1] TRUE
```

```
1 FALSE | NA
```

```
R
```

```
[1] NA
```

Special values (double)

R also has the non-finite values defined by the IEEE floating point standard (these are not unique to R): **NaN** (not a number), **Inf**, and **-Inf**.

```
1 -1 / 0
```

```
[1] -Inf
```

```
1 0 / 0
```

```
[1] NaN
```

```
1 1/0 + 1/0
```

```
[1] Inf
```

```
1 Inf - Inf
```

```
[1] NaN
```

```
1 NaN / NA
```

```
[1] NA
```

```
1 NaN * NA
```

```
[1] NA
```

These are all **double** values, but their coercion behavior is not the same as other doubles,

```
1 typeof(Inf)
```

```
[1] "double"
```

```
1 typeof(NaN)
```

```
[1] "double"
```

```
1 as.integer(Inf)
```

Warning: NAs introduced by coercion to integer r

```
[1] NA
```

```
1 as.integer(NaN)
```

```
[1] NA
```

Testing for NA, NaN, and Inf

```
1 is.na(NA)
```

```
[1] TRUE
```

```
1 is.nan(NA)
```

```
[1] FALSE
```

```
1 is.finite(NA)
```

```
[1] FALSE
```

```
1 is.na(NaN)
```

```
[1] TRUE
```

```
1 is.nan(NaN)
```

```
[1] TRUE
```

```
1 is.finite(NaN)
```

```
[1] FALSE
```

```
1 is.na(Inf)
```

```
[1] FALSE
```

```
1 is.nan(Inf)
```

```
[1] FALSE
```

```
1 is.finite(Inf)
```

```
[1] FALSE
```

Note that `is.na()` is `TRUE` for `NaN` but `is.nan()` is `FALSE` for `NA`. Comparisons involving `NA` or `NaN` always return `NA` so these predicates must be used instead of `==`.

```
1 NA == NA
```

```
[1] NA
```

```
1 NaN == NaN
```

```
[1] NA
```

```
1 c(1, NA, 3) == NA
```

```
[1] NA NA NA
```

NULL in R

NULL represents the absence of an object or value. Unlike **NA**, it has length zero and is removed when combined with an atomic vector.

```
1 typeof(NULL)
```

R

```
[1] "NULL"
```

```
1 length(NULL)
```

R

```
[1] 0
```

```
1 is.null(NULL)
```

R

```
[1] TRUE
```

```
1 c(1, NULL, 2)
```

R

```
[1] 1 2
```

```
1 list(1, NULL, 2)
```

R

```
[[1]]
```

```
[1] 1
```

```
[[2]]
```

```
NULL
```

```
[[3]]
```

```
[1] 2
```

None in Python

Base Python has no built-in missing-value type. `None` is a singleton object (of type `NoneType`) used to represent “no value,” making it closer to R’s `NULL` than to `NA`.

```
1 print(None)
```

py

None

```
1 type(None)
```

py

<class 'NoneType'>

```
1 x = None
2 x is None
```

py

True

```
1 None + 1
```

py

TypeError: unsupported operand type(s) for

```
1 [1, None, "a"]
```

py

[1, None, 'a']

Unlike `NA`, `None` does not propagate through calculations (it raises an error) and it has no type-specific variants.

Because `None` is a singleton, test for it with `is None` or `is not None` rather than `==` or `!=`. Missing values are instead

Non-finite values in Python

`nan` and `inf` are available (as `floats`) via `float()` or the `math` module. Note that division by zero is *always* an error in Python (even for floats) rather than producing `inf`.

<pre>1 1 / 0</pre> ZeroDivisionError: division by : inf	<pre>1 math.inf</pre> inf	<pre>1 math.isnan(math.nan)</pre> True
<pre>1 float("inf")</pre> inf	<pre>1 math.nan</pre> nan	<pre>1 math.isinf(-math.inf)</pre> True
<pre>1 float("nan")</pre> nan	<pre>1 math.inf - math.inf</pre> nan	<pre>1 math.nan == math.nan</pre> False

Testing is done with `math.isnan()`, `math.isinf()`, and `math.isfinite()` and, as in R, `nan` is not equal to itself (or anything else).

Exercise 1

Part 1

What is the type of the following R vectors? Explain why they have that type.

```
1 c(1, NA+1L, "C")
2 c(1L / 0, NA)
3 c(1:3, 5)
4 c(3L, NaN+1L)
5 c(NA, TRUE)
6 c(1L, NULL, NA)
```

R

Part 2

Without running the code, what is the value (and type) of each of the following Python expressions?

```
1 True + 1.5
2 "1" * 3
3 int("3") + float("1.5")
4 bool("False")
5 int(True) / True
```

py

Part 3

Considering R's four most commonly used atomic types (`logical`, `integer`, `double`, and `character`), what is the implicit conversion hierarchy from highest to lowest priority? How does this compare to Python's numeric promotion rules?

Python lists

Lists

Python lists are *ordered, mutable* sequences that can contain objects of different types (very similar to lists in R, which we will see in a few lectures).

```
1 [0, 1, 1, 0]
```

py

```
[0, 1, 1, 0]
```

```
1 [0, True, "abc"]
```

py

```
[0, True, 'abc']
```

```
1 [0, [1, 2], [3, [4]]]
```

py

```
[0, [1, 2], [3, [4]]]
```

```
1 x = [0, 1, 1, 0]
```

py

```
2 type(x)
```

```
<class 'list'>
```

```
1 len(x)
```

py

```
4
```

```
1 2 in x
```

py

```
False
```

```
1 x + [3, 4, 5]
```

py

```
[0, 1, 1, 0, 3, 4, 5]
```

Indexing

Elements are accessed using `[]` with **0-based indexing**, negative indexes count from the end, and ranges of values can be specified using slices (`start:stop` where `stop` is *exclusive*).

```
1 x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

py

```
1 x[0]
```

py

1

```
1 x[0:3]
```

py

[1, 2, 3]

```
1 x[3]
```

py

4

```
1 x[3:]
```

py

[4, 5, 6, 7, 8, 9]

```
1 x[-1]
```

py

9

```
1 x[-3:]
```

py

[7, 8, 9]

Mutability

Since lists are mutable the stored values can be changed, removed, or added to (in place).

```
1 x = [1, 2, 3, 4, 5]
```

py

```
1 x[0] = -1
2 x
```

py

`[-1, 2, 3, 4, 5]`

```
1 del x[0]
2 x
```

py

`[2, 3, 4, 5]`

```
1 x.append(7)
2 x
```

py

`[2, 3, 4, 5, 7]`

```
1 x.insert(1, -5)
2 x
```

py

`[2, -5, 3, 4, 5, 7]`

```
1 x.pop()
```

py

`7`

```
1 x
```

py

`[2, -5, 3, 4, 5]`

Assignment & identity

Assignment & copies

Ordinary R vectors use copy-on-modify semantics: after `y = x`, modifying `y` does not change `x`. Reference-style objects such as environments are exceptions.

In Python, assignment binds a *name* to an *object*: both names refer to the same object, so modifying a mutable object through one name is visible through the other.

```
1 x = c(1, 2, 3)
2 y = x
3 y[1] = 100
```

R

```
1 x = [1, 2, 3]
2 y = x
3 y[0] = 100
```

py

```
1 x
```

R

```
1 x
```

py

```
[1] 1 2 3
```

```
[100, 2, 3]
```

```
1 y
```

R

```
1 y
```

py

```
[1] 100 2 3
```

```
[100, 2, 3]
```

Shallow copies

To make an independent copy of a Python list, use `.copy()`:

```
1 x = [1, 2, 3]
2 y = x.copy()
3 y[0] = 100
4 x
```

py

[1, 2, 3]

A list's `.copy()` method copies the outer list, but nested mutable objects remain shared.

```
1 x = [[1, 2], [3, 4]]
2 y = x.copy()
3
4 y[0][0] = 100
5 x
```

py

[[100, 2], [3, 4]]

```
1 y
```

py

[[100, 2], [3, 4]]

Identity vs equality

Because multiple Python names can refer to the same object, Python distinguishes between *identity* (`is` - are these the same object?) and *equality* (`==` - do they have the same value?). The `id()` function returns an integer representing an object's unique identity.

```
1 x = [1, 2, 3]
2 y = x
3 z = x.copy()
```

```
1 x is y
```

py

True

```
1 x is z
```

py

False

```
1 x == z
```

py

True

```
1 id(x)
```

py

4645915968

```
1 id(y)
```

py

4645915968

```
1 id(z)
```

py

4645913920

R's `identical()` tests strict value equality (including type and attributes), not object identity. For example, `identical(1L, 1)` is `FALSE` while `1L == 1` is `TRUE`. With ordinary copy-on-modify objects, identity rarely matters in R.

Comparing R & Python

Scalars vs vectors

For numeric data, the biggest conceptual difference is that R represents even a single atomic value as a vector, while base Python distinguishes individual numeric values from containers. This affects how arithmetic on collections behaves,

```
1 x = c(1, 2, 3)
2 length(x)
```

R

```
[1] 3
```

```
1 x * 2
```

R

```
[1] 2 4 6
```

```
1 x + x
```

R

```
[1] 2 4 6
```

```
1 length(1)
```

R

```
[1] 1
```

```
1 x = [1, 2, 3]
2 len(x)
```

py

```
3
```

```
1 x * 2
```

py

```
[1, 2, 3, 1, 2, 3]
```

```
1 x + x
```

py

```
[1, 2, 3, 1, 2, 3]
```

```
1 len(1)
```

py

```
TypeError: object of type 'int' has no len
```

Next week we will see how [numpy](#) arrays bring R-style vectorized operations to Python.

Type summary

Concept	R	Python
boolean	<code>logical (TRUE, FALSE)</code>	<code>bool (True, False)</code>
integer	<code>integer (1L)</code>	<code>int (1)</code>
real	<code>double (1, 1.5)</code>	<code>float (1.0, 1.5)</code>
complex	<code>complex (1i)</code>	<code>complex (1j)</code>
text	<code>character ("a")</code>	<code>str ("a")</code>
type of	<code>typeof()</code> , <code>class()</code> , <code>is.*()</code>	<code>type()</code> , <code>isinstance()</code>
conversion	<code>as.integer()</code> , <code>as.character()</code> , ...	<code>int()</code> , <code>str()</code> , ...
missing	<code>NA</code> (typed)	library-specific sentinels
absent / null	<code>NULL</code>	<code>None</code>
non-finite	<code>Inf</code> , <code>-Inf</code> , <code>NaN</code>	<code>math.inf</code> , <code>-math.inf</code> , <code>math.nan</code>
containers	atomic vectors, lists	<code>list</code> , ...

Operator summary

Operation	R	Python
exponent	<code>^</code>	<code>**</code>
integer division	<code>%/%</code>	<code>//</code>
modulus	<code>%%</code>	<code>%</code>
and / or / not	<code>&, , !</code>	<code>and, or, not</code>
value equality	<code>==, identical()</code>	<code>==</code>
object identity	<code>—</code>	<code>is</code>
membership	<code>%in%</code>	<code>in</code>
string concat	<code>paste0()</code>	<code>+</code>
interpolation	<code>glue::glue()</code>	<code>f-strings</code>
raw strings	<code>r"(...)"</code>	<code>r"..."</code>

These are not exact equivalents: R's `&` and `|` are vectorized and evaluate both sides, whereas Python's `and` and `or` short-circuit and return one of their operands. R also has the short-circuit operators `&&` and `||`, which we will discuss with

Indexing

R uses **1-based** indexing and negative indexes *exclude* elements, Python uses **0-based** indexing and negative indexes count from the *end*.

```
1 x = c("a", "b", "c", "d")
2 x[1]
```

R

```
[1] "a"
```

```
1 x[4]
```

R

```
[1] "d"
```

```
1 x[-1]
```

R

```
[1] "b" "c" "d"
```

```
1 x[2:3]
```

R

```
[1] "b" "c"
```

```
1 x = ["a", "b", "c", "d"]
2 x[0]
```

py

```
'a'
```

```
1 x[3]
```

py

```
'd'
```

```
1 x[-1]
```

py

```
'd'
```

```
1 x[1:3]
```

py

```
['b', 'c']
```

Note that R's **2:3** *includes* both endpoints while Python's **1:3** slice *excludes* the stop value - both select the 2nd and 3rd elements here.

Exercise 2

Without running the code, predict the output (or error) of each of the following in *both* R and Python.

```
1 x = c(1, 2, 3)
2 y = x
3 y[1] = "a"
4 x
5 y
```

R

```
1 x = [1, 2, 3]
2 y = x
3 y[0] = "a"
4 x
5 y
```

py

```
1 TRUE + TRUE == 2
2 "2" == 2
3 as.integer("2") + 1L
4 0.1 + 0.2 == 0.3
```

R

```
1 True + True == 2
2 "2" == 2
3 int("2") + 1
4 0.1 + 0.2 == 0.3
```

py

```
1 c(1, 2.5, NA)
2 sum(c(1, 2.5, NA))
```

R

```
1 [1, 2.5, None]
2 sum([1, 2.5, None])
```

py

Takeaways

- R's atomic vectors are homogeneous and vectorized; base Python uses individual objects and general-purpose containers.
- Types determine which operations are valid and when coercion occurs.
- `NA`, `NULL`, `None`, and `nan` represent different ideas.
- R vectors usually copy on modify; Python mutable objects can be shared by multiple names.